

SKYSCANNER API REPORT

Prepared By : V Ajay Krishnan

1.Objective

The main objective of this API testing is to verify the Post endpoints are working and performing different assertion to validate the fetched data.

The testing checks:

- Whether a new post can be created successfully
- Whether the response contains correct data
- Whether the response follows the expected JSON schema
- Both request and response validations are performed to ensure the API behaves correctly.

2.Environment & setup

- Language Used: Java
- Framework: Rest Assured
- Testing Framework: JUnit 5

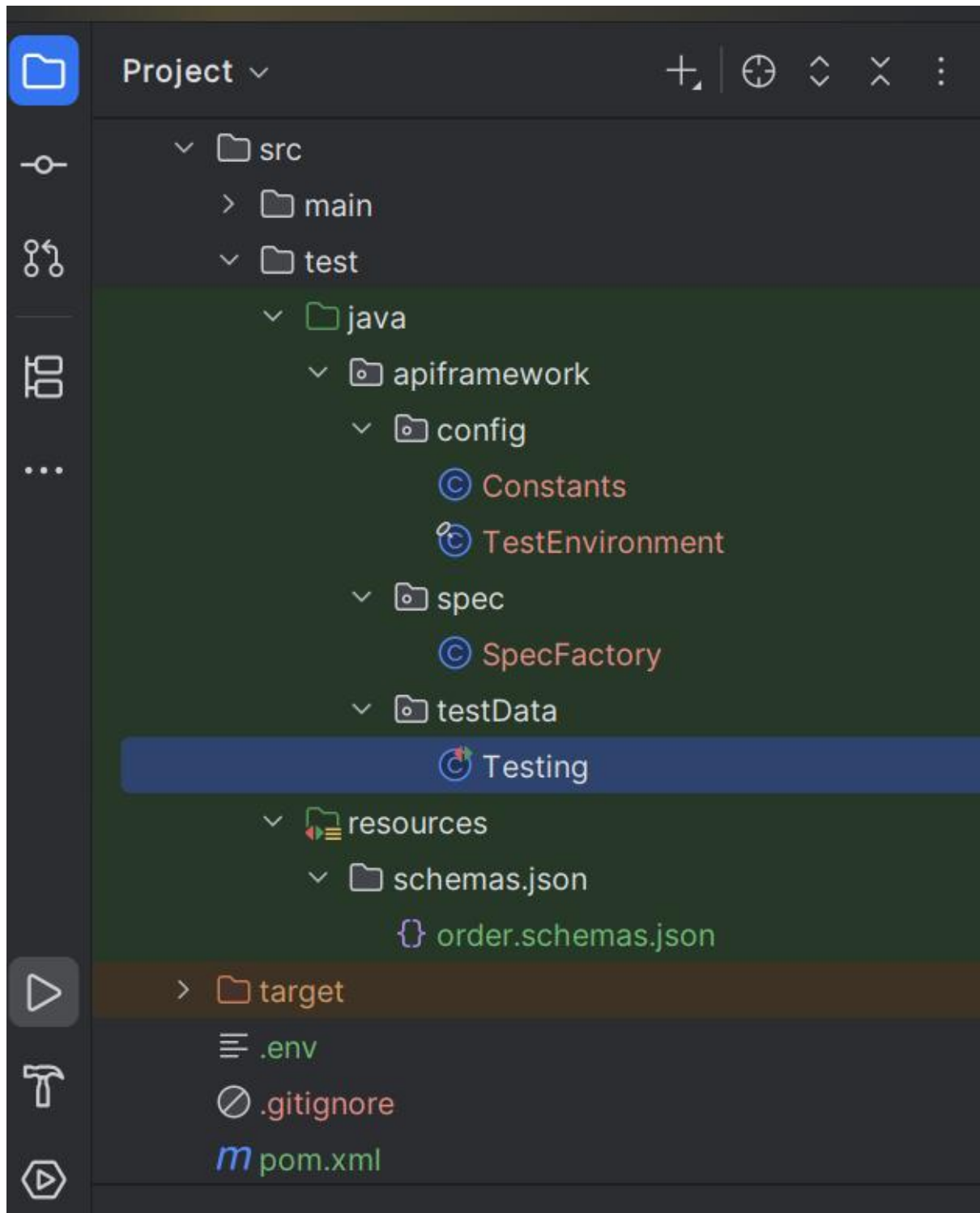
- Build Tool: Maven
- OS: Windows
- API URL: <https://jsonplaceholder.typicode.com>

3. Framework design

The framework follows a simple OOPS structure:

- Config (Constants):
Stores the Base URL, which is read from the .env file using environment configuration.
- Spec (SpecFactory):
Contains reusable request and response specifications for POST requests.
- Test Data (Testing class):
Contains the test case for validating the POST API along with assertions and schema validation.

4. Project structure



- config → Stores environment setup and Base URL
- spec → Contains request and response specifications
- testData → Contains test class for POST API validation
- schemas.json → Contains JSON schema for response validation
- .env → Stores Base URL configuration

5. Test Execution Command

mvn test

6. STEPS PERFORMED

POST Request

- A new post is created using the /posts endpoint
- Request body includes:
 - title
 - body
 - userId
- Validations performed:
 - Status code is 201
 - Response contains a generated id

Response fields match request data:

- title
- body
- userId

Fields are not empty

Response is validated using JSON schema
(order.schemas.json)

7.Result

The POST API test was executed successfully:

- Post creation passed
- Response validation passed
- Schema validation passed
- Validated all data

All assertions were validated correctly.

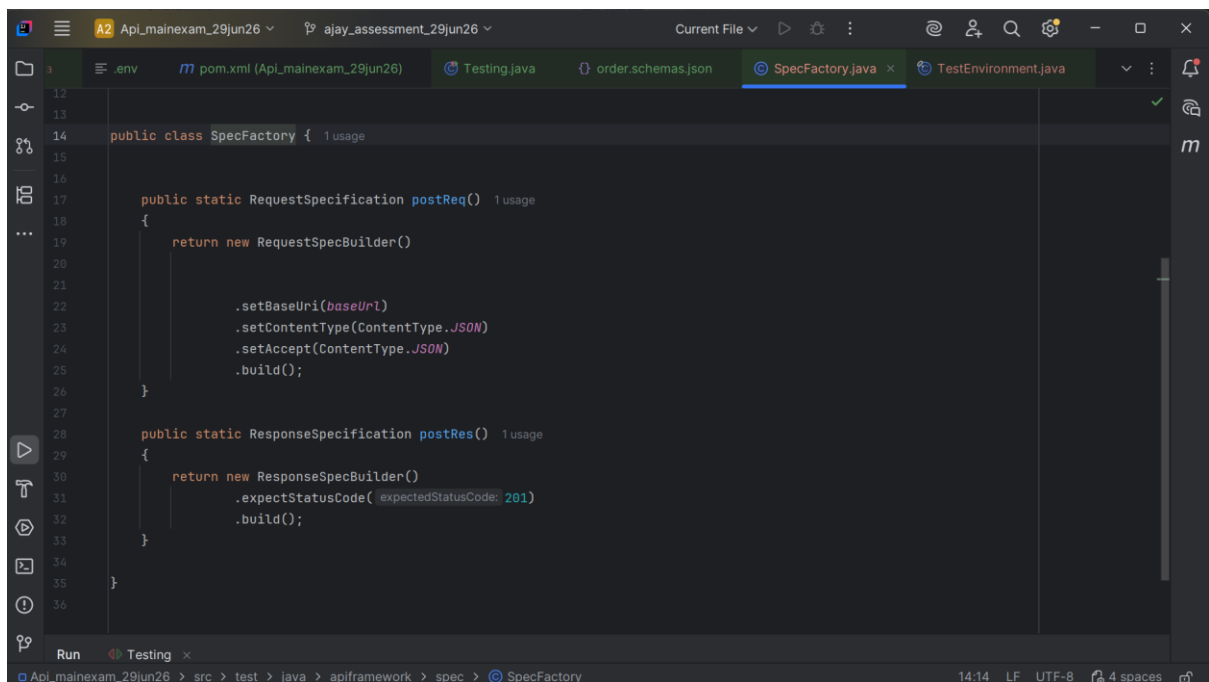
8.Conclusion

The POST API is working as expected.

The response data is accurate and follows the defined schema.

The framework uses reusable specifications, making the test code clean and easy to maintain.

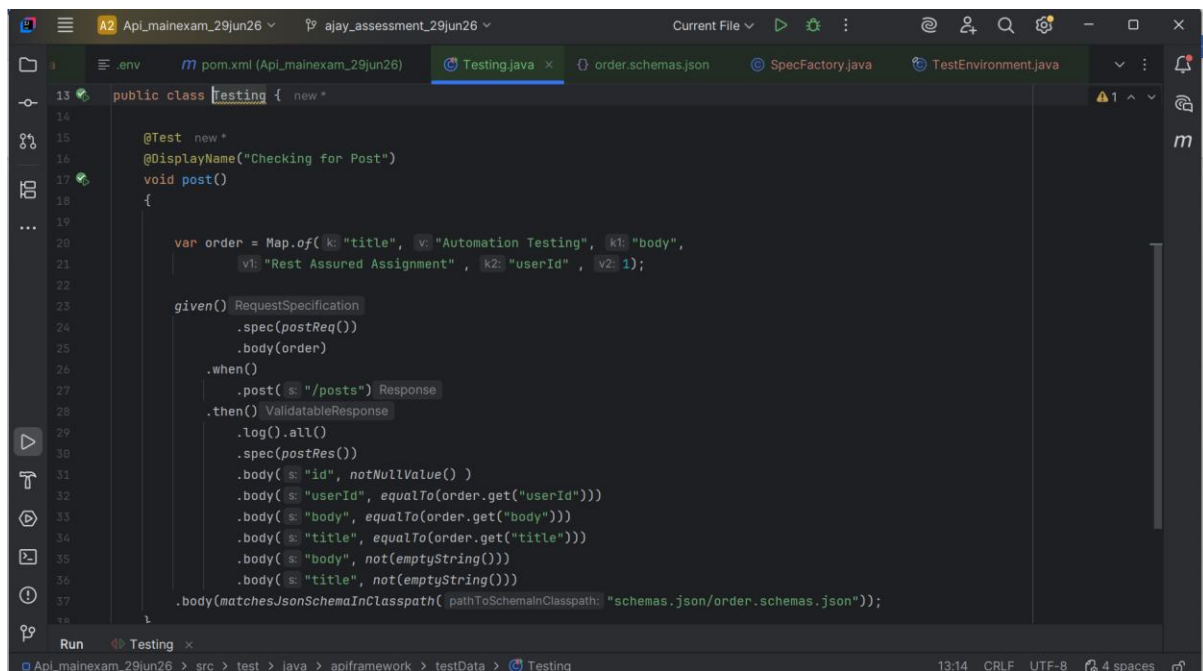
8.Codes



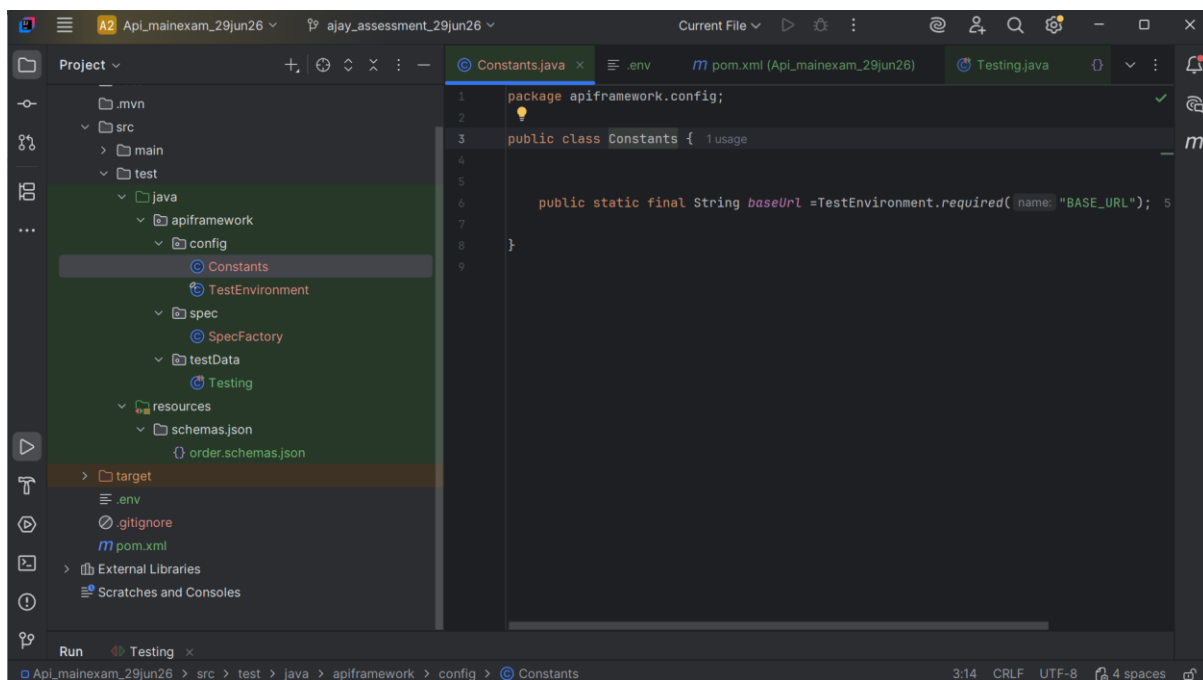
```
12
13
14 public class SpecFactory { 1 usage
15
16     public static RequestSpecification postReq() 1 usage
17     {
18         return new RequestSpecBuilder()
19
20         .setBaseUrl(baseUrl)
21         .setContentType(ContentType.JSON)
22         .setAccept(ContentType.JSON)
23         .build();
24     }
25
26     public static ResponseSpecification postRes() 1 usage
27     {
28         return new ResponseSpecBuilder()
29         .expectStatusCode( expectedStatusCode: 201)
30         .build();
31     }
32
33
34
35
36
```

Run Testing x

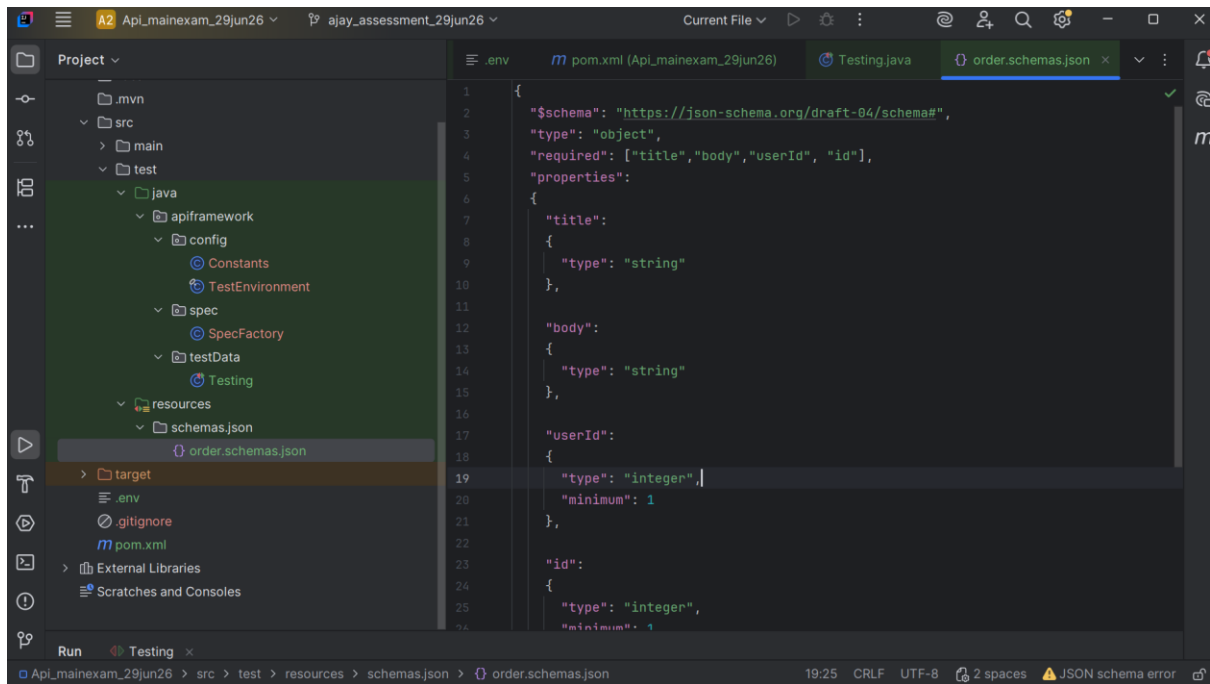
Api_mainexam_29jun26 > src > test > java > apiframework > spec > SpecFactory 14:14 LF UTF-8 4 spaces



```
13 public class Testing {
14
15     @Test
16     @DisplayName("Checking for Post")
17     void post()
18     {
19
20         var order = Map.of( k: "title", v: "Automation Testing", k1: "body",
21                             v1: "Rest Assured Assignment", k2: "userId", v2: 1);
22
23         given() RequestSpecification
24             .spec(postReq())
25             .body(order)
26             .when()
27                 .post( "/posts") Response
28             .then() ValidatableResponse
29                 .log().all()
30                 .spec(postRes())
31                 .body( "id", notNullValue() )
32                 .body( "userId", equalTo(order.get("userId")))
33                 .body( "body", equalTo(order.get("body")))
34                 .body( "title", equalTo(order.get("title")))
35                 .body( "body", not(emptyString()))
36                 .body( "title", not(emptyString()))
37                 .body(matchesJsonSchemaInClasspath( pathToSchemaInClasspath: "schemas.json/order.schemas.json"));
```



```
1 package apiframework.config;
2
3 public class Constants {
4
5     public static final String baseUrl = TestEnvironment.required( name: "BASE_URL");
6
7 }
8
9
```



9.Evidence

Project ▾

Run Testing x

Testing (apiframework.test 3 sec 485 ms) 1 test passed 1 test total, 3 sec 485 ms

Checking for Post 3 sec 485 ms

Reporting-Endpoints: heroku-nel="[https://nel.heroku.com/reports?s=8FMQCCrISjaY0NHQIZbZQhKeJWoE9EIYUV6%](https://nel.heroku.com/reports?s=8FMQCCrISjaY0NHQIZbZQhKeJWoE9EIYUV6%22)"

Server: cloudflare

Vary: Origin, X-HTTP-Method-Override, Accept-Encoding

Via: 2.0 heroku-router

X-Content-Type-Options: nosniff

X-Powered-By: Express

X-Ratelimit-Limit: 1000

X-Ratelimit-Remaining: 999

X-Ratelimit-Reset: 1782732990

Cf-Cache-Status: DYNAMIC

CF-RAY: a134a2066b880803-MAA

alt-svc: h3=":443"; ma=86400

```
{
  "body": "Rest Assured Assignment",
  "userId": 1,
  "title": "Automation Testing",
  "id": 101
}
```

Process finished with exit code 0

Api_mainexam_29jun26 > src > test > java > apiframework > testData > Testing 3:14 CRLF UTF-8 4 spaces